BEDROCK ARCHITECTURE

Bedrock C ABI

C Language Binding and Far-Pointer Calling Convention

Draft

CONTENTS

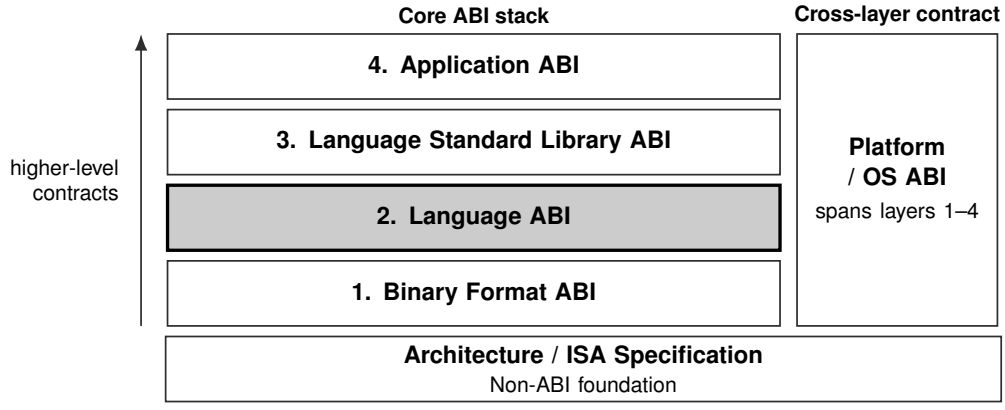
LIST OF TABLES

LIST OF FIGURES

1 SCOPE AND CONFORMANCE

This document defines the Bedrock C data model, register convention, calling convention, far-pointer representation and calls, aggregate and variadic rules, compiler-runtime helper ABI, and C memory-model lowering. It depends on the Bedrock ELF ABI and the active platform/runtime environment ABI. Source syntax and semantics for target-specific C extensions are defined separately.

1.1 Contract Position



Current document: Bedrock C ABI. The shaded block identifies its primary contract position. The Platform / OS ABI spans and constrains the four core ABI layers; the ISA specification is their non-ABI architectural foundation.

1.2 Profile

Profile: `bedrock-c`

Defined extended type family: `bedrock-c-far`

2 TERMINOLOGY

These terms are specific to the C binding. ELF and address-space terms keep the meanings defined by the Bedrock ELF ABI and the architecture manual.

object pointer A 64-bit pre-segment virtual address that designates an object or one-past an object according to the C abstract machine.

function pointer A 64-bit pre-segment code address naming a near-call entry point. It is not a CS:PC pair, far descriptor, or runtime-private veneer descriptor.

far data pointer A 128-bit pointer containing a canonical pre-segment virtual address and a disabled or bounds-only segment image.

far function pointer A 128-bit function pointer containing a pre-segment far-call entry address and a disabled or bounds-only `CS` image.

far-call entry An `LCALL`/`LRET` entry point whose incoming stack contains a 16-byte `PC:CS` return frame.

caller-saved register A register whose value may be overwritten by a call unless the caller saves it before the call.

callee-saved register A register whose incoming value must be preserved by the called function if the function uses it.

sret buffer Caller-allocated storage used to receive a return value that is too large or unsuitable for the normal return registers.

addressable by-value argument A by-value aggregate argument that is passed through caller-owned stack storage because the callee may need an address for the object.

3 DATA MODEL

Model: LP64

Byte Order: little endian

Internal SP: 8-byte aligned

Function Entry: 16-byte aligned

Before CALL: SP mod 16 = 8

Aggregate Maximum Alignment: 16 bytes

3.1 Scalar Types

Table 3-1. Reference C Data Model

Type	Bits	Align
`_Bool`	8	1
`char`	8	1
`signed char`	8	1
`unsigned char`	8	1
`short`	16	2
`int`	32	4
`long`	64	8
`long long`	64	8
`__int128`	128	16
`unsigned __int128`	128	16
`ordinary pointer`	64	8
`far data pointer`	128	16
`far function pointer`	128	16
`size_t`	64	8
`ptrdiff_t`	64	8
`wchar_t`	32	4
`char16_t`	16	2
`char32_t`	32	4
`float`	32	4
`double`	64	8
`long double`	64	8
`float _Complex`	64	4
`double _Complex`	128	8
`long double _Complex`	128	8

3.2 Scalar Type Semantics

The widths and natural alignments of scalar types are given in Table 3-1. The following rules define the type identities and representations that are not apparent from those two values.

Integer types and aliases. Plain `char` is signed. `size_t` is an alias for `unsigned long`, and `ptrdiff_t` is an alias for `long`. `wchar_t` is a signed 32-bit integer; `char16_t` and `char32_t` are unsigned 16-bit and 32-bit integers, respectively. In the absence of a separate enum extension, an enum has the representation, size, and alignment of `int`.

A stored `_Bool` has the integer value zero or one. False is encoded with every object-representation bit clear; true is encoded with bit 0 set and every other bit clear.

Ordinary pointers. An ordinary object or function pointer is a 64-bit, 8-byte-aligned canonical pre-segment virtual address. Object and function pointers remain distinct C type categories even though their baseline representations have the same size and alignment. The null pointer has numeric address zero and an all-zero 64-bit object representation. Every ordinary-pointer result with numeric address zero is the null pointer.

128-bit integers. The types `__int128` and `unsigned __int128` are 16-byte, 16-byte-aligned integers. Their little-endian object representation places the low 64-bit word at the lower address. When the calling convention assigns one to a GENERAL-PAIR, the low word occupies the lower-numbered even register and the high word occupies the following odd register. A returned 128-bit integer therefore uses `R0` for the low word and `R1` for the high word. Its stack slot is 16-byte aligned.

An ordinary 128-bit load or store need not be atomic and may use two 64-bit memory operations. The 128-bit integer types are not supported as C atomic objects in the baseline profile.

Floating-point profile. The `bedrock-c` profile requires the base floating-point extension. Floating-point helpers may replace unavailable operations only when the `F0`–`F15` register calling convention remains available.

Long double. `long double` is a distinct C type with the IEEE-754 binary64 object representation, 8-byte size, and 8-byte alignment. It uses the floating-point argument class and returns in `F0`, making its call representation ABI-compatible with `double`. In a variadic call it occupies one 16-byte slot containing an 8-byte binary64 value without widening.

Complex types. A complex object contains its real component first and its imaginary component second, each using the corresponding real type's representation. Therefore `float _Complex` has size 8 and alignment 4, while `double _Complex` and `long double _Complex` have size 16 and alignment 8.

3.3 Far Pointer Representations

Far data and function pointers are 16-byte, 16-byte-aligned, little-endian two-word scalars with the common object representation shown below.

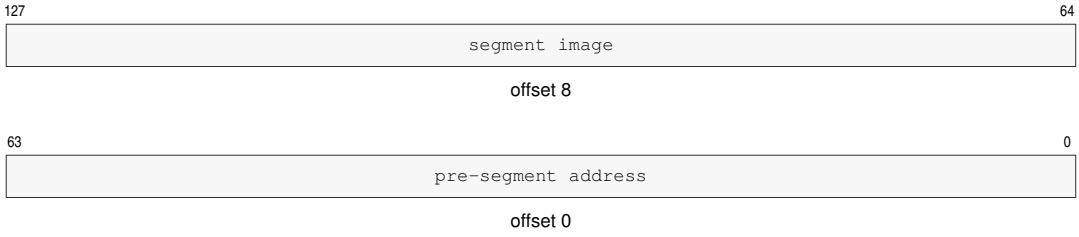


Figure 3-1. Far Pointer Object Representation

Far data pointers interpret the two words as a canonical virtual byte address and data-segment image; far function pointers interpret them as a canonical far-call entry address and `CS` image.

For either pointer kind, an enabled image must be validated and bounds-only, and a disabled image must be canonical. A far pointer is null exactly when its address word is zero. Its image word remains part of the representation and must still satisfy the ordinary image-validity rule. The all-zero pair is the canonical encoding produced by explicit null construction, but a zero-address far value with another valid image is the same null pointer value with a different raw encoding. Translated-window images are not ABI-valid stored, passed, returned, relocated, or GOT-resolved far pointers. For an explicitly translated source, the segment base is added to the EA modulo 2^{64} and the image is converted to bounds-only mode. The converted image is retained for every sum, including a zero sum produced by wraparound. A zero sum denotes null without changing that image word.

The unsigned raw carriers `__bedrock_far_uintptr_t` and `__bedrock_far_func_uintptr_t` preserve both words. Far pointer metadata is forgeable address metadata, not an unforgeable capability. Both carrier names are aliases of `unsigned __int128`; the separate names document data-pointer and function-pointer use but do not create distinct C types. Conversion through a matching carrier preserves a zero-address value's image word; it does not replace that representation with the all-zero pair.

3.4 Aggregate Layout

Scalar and pointer object representations are defined in Sections 3.1 through 3.3. Aggregate layout applies those component sizes and alignments as follows:

- Struct fields are laid out in declaration order with natural alignment up to the aggregate maximum.
- A union's alignment is the greatest alignment required by any member. Its size is the greatest ABI size of any member, rounded up to a multiple of the union alignment.
- Arrays store elements contiguously with each element using the ABI size of its type.
- Aggregate object size is rounded up to the aggregate object's alignment.

Bit-fields. A bit-field allocation unit has the size and alignment of its declared base type. Within the little-endian unit, allocation begins at the low bit. A field never crosses an allocation-unit boundary, and only consecutive fields whose base types are identical after typedef expansion may share a unit. A base-type change or a field that does not fit closes the current unit and

begins a new naturally aligned unit. A field width may not exceed the representation width of its base type.

An externally visible bit-field base type shall be `_Bool`, `signed int`, `unsigned int`, or a typedef of one of those types. A plain `int` bit-field has the value range and representation of `signed int`. Other base types are target extensions and may not cross an external ABI boundary without a separately documented compatible extension ABI.

An unnamed nonzero-width field consumes space normally. A zero-width field closes the current unit and advances layout to the next boundary for its base type without adding storage of its own. Ordinary aggregate alignment, tail-padding, and size-rounding rules still apply after bit-field allocation.

Flexible arrays and excluded extensions. A flexible array member contributes no element storage to `sizeof` its containing structure. Its offset is aligned for the element type, and that element alignment participates in the structure's alignment calculation.

Packed aggregates and `#pragma pack` are not baseline ABI facilities. A compiler may expose them only as an extension, and such a type may not cross an external ABI boundary without a separately documented compatible extension ABI. Empty structures and zero-length arrays are likewise excluded from the baseline profile.

4 REGISTER CONVENTION

This section defines which architectural state survives an ordinary C call. Argument and result locations are defined by the assignment procedure in Section 5; they are intentionally not duplicated here.

4.1 Register Preservation

The caller shall preserve any live value held in a volatile register before a call. A callee that modifies a nonvolatile register shall restore its exact incoming value before returning. Registers not listed as nonvolatile carry no value across the call unless another rule in this ABI explicitly assigns the call result to them.

Table 4-1. C Call Preservation Sets

State	Volatile across a call	Preserved by the callee
General registers	`R0`–`R7`	`R8`–`R15`
Floating-point registers	`F0`–`F7`	`F8`–`F15`
Condition state	`FLAGS`	none
Segment state	`GS1`–`GS5`	`CS`, `DS`, `SS`, `GS0`

A callee may preserve floating-point registers with `FPUSHP` and `FPOPP` or with any sequence having the same observable effect. Each instruction operates on one canonical pair. The nonvolatile pairs are index 0 for `F14:F15`, index 1 for `F12:F13`, index 2 for `F10:F11`, and index 3 for `F8:F9`. When several pairs are saved, a normal prologue pushes them in increasing pair-index order and the matching epilogue pops them in decreasing pair-index order. A callee need only save the pair containing each nonvolatile register it modifies. `R15` is an ordinary nonvolatile register unless the function elects to use it as a frame pointer.

4.2 Stack and Control State

SP is the architectural stack pointer, not a general C temporary. A callee may move it while executing but shall restore the entry value before returning; Section 5 defines the additional alignment and frame rules. PC is the architectural instruction pointer and is visible to C code only as a control-flow target. The call and return instructions, rather than a preservation class, define its transition across a call. Condition codes in FLAGS are volatile and may be clobbered by every callee.

4.3 Segment Context

An ordinary C call preserves the exact incoming images of CS, DS, SS, and GS0. The first three establish the code, ordinary-data, and stack contexts. GS0 establishes the TLS context and may use translated-window mode even though translated-window CS, DS, and SS are excluded below.

GS1 is caller-saved scratch and carries a returned far pointer's segment image. GS2 through GS5 are caller-saved far-pointer argument segment registers. No incoming value in these five registers is preserved across a near or far call. Named arguments define the callee-entry values of GS2 through GS5; a far-pointer result defines the caller-continuation value of GS1.

4.4 Far Segment Register Channels

The low word of a far data pointer is a pre-segment address. A named register argument arrives with its address in an Rn register and its validated segment image already established in the corresponding GS2 through GS5 argument register. Generated code may use that pair directly in a segment-qualified effective address.

A far pointer loaded from memory, held locally, or received on the stack uses GS1 as scratch: generated code writes the high word to GS1 before the dependent access and uses the low word through a GS1-qualified effective address. A call or other operation that may clobber the selected caller-saved GS register requires the image to be re-established before the next dependent access.

An ABI-valid image is disabled or bounds-only, so the selected GS register contributes no base translation. Segment-image validation precedes the dependent memory access; bounds failure raises PAGE_FAULT, and page translation occurs after segment pre-translation.

4.5 Ordinary C Address-Space Invariants

Ordinary object and function pointers are canonical pre-segment virtual addresses. Conforming C execution maintains all of the following invariants:

- CS, DS, and SS are disabled with a zero mantissa or are enabled in bounds-only mode. Translated-window images for these three registers are outside the baseline C ABI.
- Using an ordinary pointer as an effective address preserves its numeric value across segment processing.
- DS is disabled or its bounds contain every address that may be dereferenced through an ordinary object pointer, including materialized stack and TLS addresses.
- CS permits near-call reachability for every ordinary function entry in the active linkage domain, and SS covers the active stack.

- Pre-segment virtual address zero is not assigned to a C object or function entry. Every C object and its one-past address must be representable as nonzero 64-bit addresses; no object may have mathematical one-past address 2^{64}.

The active OS ABI selects the initial disabled or bounds-only images and owns loader mapping policy. It shall preserve pointer numeric identity, ordinary pointer coverage, near-call reachability, and active-stack coverage whenever it changes those images.

4.6 Status Registers

`STATUS` is not part of the C abstract machine; ordinary C code accesses it only through target-specific intrinsics or runtime code. Low-level access to `FSTATUS` and `FFLAGS` uses `<bedrockfpuintrin.h>`. `FSTATUS` is callee-saved: a function returns with the incoming rounding mode, trap enables, and other control fields unless its documented interface changes the floating-point environment. `FFLAGS` is cumulative floating-point status and is caller-clobbered; floating-point operations in a callee may set accrued exception flags. An interface that requires a saved environment uses its floating-point environment runtime contract explicitly.

5 CALLING CONVENTION

This section is normative. A caller and callee conform when they independently apply the classification and assignment procedure below and therefore agree on every argument location, return location, and stack boundary. Register lists are summaries; they do not replace the assignment procedure.

5.1 Stack and Calls

The stack grows downward. An architectural `CALL` pushes an 8-byte return address at `[SP]`, and callers align `SP` before the call so callee entry observes 16-byte alignment. Between ABI call boundaries, a function may adjust `SP` while keeping it aligned to at least 8 bytes. Immediately before `CALL`, the stack pointer satisfies SP modulo 16 equals 8 before return address push. After `CALL` pushes the return address, it satisfies SP modulo 16 equals 0 at callee entry.

Each object observes its declared type alignment. In particular, relaxing the internal stack-pointer invariant does not relax the 16-byte alignment of `__int128` objects or other objects whose declared alignment is 16 bytes. The ABI defines no red zone and no fixed outgoing-argument area. A callee that dynamically realigns the stack restores the incoming stack pointer before returning.

The first stack argument begins at `[SP+16]` at callee entry. The eight bytes at `[SP+8]` are alignment padding and carry no argument value. Each subsequent stack argument occupies the next 16-byte slot.

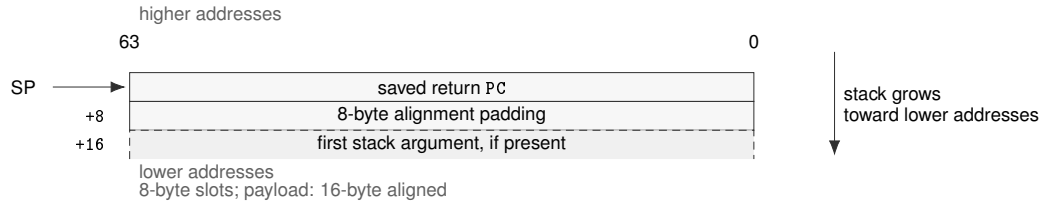


Figure 5-1. Near-call stack frame at callee entry

5.2 Far Calls and Returns

A far function pointer supplies a pre-segment code address and a canonical disabled or bounds-only `CS` image. Direct calls to far-native entries and indirect calls through far function pointers use `LCALL`; the callee returns with `LRET`. Argument classification, floating-point assignment, return values, structure returns, and callee-saved registers are otherwise identical to the ordinary convention.

The caller aligns `SP` to 0 modulo 16 before `LCALL`. The instruction creates a 16-byte return frame, so the callee also observes `SP` modulo 16 equal to zero at entry. The first stack argument begins at `[SP+16]` without the near-call padding slot.

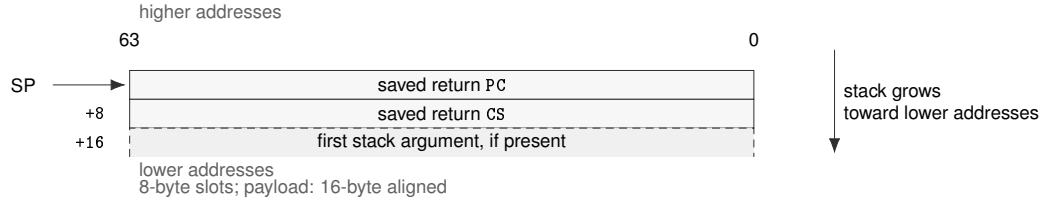


Figure 5-2. Far-call stack at callee entry

`LCALL` changes `CS` to the pointer image and preserves `DS`, `SS`, and `GS0`. `GS1` through `GS5` are caller-saved. The callee therefore uses the caller's ordinary data, stack, and TLS contexts while receiving far-pointer segment arguments in `GS2` through `GS5`. Changing `DS` or `SS` requires an OS/runtime call gate and is not a C far call.

No fixed general register is reserved for the target operands. Because `LCALL` takes its new-CS image from an Rn operand, an image held in a GS register is first copied with `RDSEG`. The compiler may choose the Rn temporary, preserving any nonvolatile register it uses. The callee restores its entry `SP` before `LRET`.

A compatible far-to-far tail transfer may use `LJMP` only after restoring the entry stack pointer and when argument layout and return convention match. Near-to-far and far-to-near tail calls are forbidden. A nonreturning far transfer may use `LJMP`.

Far-frame unwind metadata records saved `PC` and `CS` as separate words and identifies the 16-byte return frame. A far-aware jump environment records `PC`, `CS`, `SP`, and the baseline callee-saved set. A near-only jump environment cannot target a far frame.

5.3 Frame Pointer

Frame pointers are omitted by default. When a function elects to keep a frame pointer, it uses `R15`. R15 is callee-saved and is used as the frame pointer when a function elects to maintain one.

5.4 DWARF Register Numbering and Unwind

Table 5-1. Bedrock DWARF Register Numbers

Numbers	Registers	Status
0..15	`R0..R15`	assigned
16	`SP`	assigned
17	`PC`	assigned; CIE return-address register
18	`FLAGS`	assigned
19	`STATUS`	assigned
20..31	—	reserved
32..47	`F0..F15`	assigned
48	`CS`	assigned
49	`DS`	assigned
50	`SS`	assigned
51..56	`GS0..GS5`	assigned
57	`FFLAGS`	assigned
58	`FSTATUS`	assigned
59 and greater	—	reserved

At near-call entry, the canonical frame address is entry `SP+8`; saved `PC` is at CFA minus 8 and `CS` has a same-value rule. At far-call entry, the CFA is entry `SP+16`; saved `PC` is at CFA minus 16 and saved `CS` is at CFA minus 8. Standard DWARF CFI rules recover both registers. Signal and architectural event-frame unwind conventions belong to the active OS ABI.

5.5 Argument Classification

Classification occurs after the C language has determined the effective type of the argument. A named argument to a prototyped function is placed in one of six ABI classes:

GENERAL Integer types no wider than 64 bits and ordinary object or function pointers. One general-register location is required.

GENERAL-PAIR Signed and unsigned `__int128`. Two consecutive general registers beginning at an even-numbered register are required.

FAR Far data and far function pointers. One general register carries the address and one of `GS2` through `GS5` carries the segment image.

FLOAT `float`, `double`, and `long double`. One floating-point register is required.

FLOAT-PAIR `float _Complex`, `double _Complex`, and `long double _Complex`. Two consecutive floating-point registers are required, with no even-register alignment.

INDIRECT Every structure or union argument, regardless of size or member composition. The caller creates a 16-byte-aligned by-value copy and passes the address of that copy as one GENERAL value.

Signed GENERAL values narrower than 64 bits are sign-extended when assigned to a register. Unsigned values and `_Bool` are zero-extended. Plain `char` follows its signed ABI type. A stack slot contains the effective type's object representation at its low address; bytes after that representation are unspecified.

5.6 Assignment Procedure

The caller shall apply the following procedure. The callee may rely on the result without inspecting any unused register or padding byte.

1. If the result uses an sret buffer, assign its address to `R0` and initialize the general-register cursor to 1. Otherwise initialize it to 0. Initialize the floating-point cursor to 0 and the far-segment cursor to `GS2`.
2. Visit arguments in source order. GENERAL and INDIRECT arguments consume the register named by the current general cursor and then advance that cursor. FLOAT arguments analogously consume `F0` through `F7` using the independent floating-point cursor.
3. For a FLOAT-PAIR argument, use the current floating-point register for the real component and the next register for the imaginary component, then advance the cursor by two. No even-register alignment is applied. If both registers are not available, place the complete complex value in one stack slot and mark the floating-point class exhausted.
4. For a GENERAL-PAIR argument, round the general cursor upward to an even register number. If that register and its successor both exist, place the low 64 bits in the even register, the high 64 bits in the odd register, and advance the cursor past the pair. A skipped odd register remains unused.
5. For a FAR argument, use the current general register for the pre-segment address and the current far-segment register for the segment image, then advance both cursors. No even-register alignment is applied. If either cursor has no location, place the complete 16-byte pointer in one stack slot and consume neither remaining cursor.
6. If a non-FAR argument cannot be placed completely in the remaining registers of its class, place the complete argument in one stack slot and mark that register class exhausted. Later arguments of that class also use stack slots; they do not backfill a register hole. An argument is never split between registers and the stack.
7. Unnamed variadic arguments and every argument of an unprototyped call are forced to stack slots and do not consume register cursors. Apply the default argument promotions before assigning those slots.
8. Assign stack slots to stack-bound arguments in source order beginning at `[SP+16]`. Consequently, a caller that constructs the outgoing area by stores or pushes does so from the rightmost stack-bound argument toward the leftmost. No register home slots are reserved.

The general, floating-point, and far-segment resources exhaust independently. Exhausting GS2 through GS5 sends later FAR arguments to the stack but does not prevent an ordinary GENERAL argument from using a remaining Rn register. A FAR argument spilled because either resource is unavailable does not consume a location from the other resource. A stack-assigned or unnamed variadic far pointer occupies one 16-byte, 16-byte-aligned slot and is never split.

Only scalar far pointers assigned through the FAR class cross an ABI call boundary in canonical, validated form. Loading a register-assigned image with WRSEG performs this validation. Before a stack-assigned, variadic, or unprototyped scalar far pointer is published, the caller validates its image through scratch GS1. Any failed validation raises INVALID_CONTROL_STATE before the control transfer.

An aggregate is transferred as its object representation and is not recursively inspected for far-pointer encodings. A contained far-pointer representation may therefore cross a call boundary inside an aggregate. It is validated when extracted for scalar return, dereference, call, or another operation that requires a valid far pointer. Invalid far values may be copied and compared wherever the language extension permits representation-preserving operations.

5.7 C Return Values

Scalar results use their natural register class. A `__int128` result places its low word in R0 and high word in R1. A structure or union of at most 16 bytes is returned as raw object bytes: increasing object addresses map first to R0 and then to R1. Bytes beyond the object size and padding bytes whose values are not defined by C are unspecified.

A complex result places its real component in F0 and its imaginary component in F1.

For a larger aggregate, the caller allocates a 16-byte-aligned result buffer and passes its address in R0 before ordinary argument assignment. The callee stores the result in that buffer and returns the same address in R0.

A far data or function pointer result uses GS1:R0, with its pre-segment address in R0 and validated segment image in GS1. A failed GS1 write raises INVALID_CONTROL_STATE before the return transfer.

Table 5-2. C Return Register Quick Reference

Result Class	Register Rule
General scalar	R0
Floating-point scalar	F0
`long double` scalar	F0
Complex scalar	real component in F0, imaginary component in F1
`__int128` scalar	R1:R0
Far data or function pointer	address in R0, segment image in GS1
Small aggregate	R1:R0
Large aggregate	sret pointer in R0, result pointer returned in R0

5.8 Aggregate Passing

For every by-value aggregate argument, the caller allocates distinct storage, aligns its start to 16 bytes, initializes it with the argument value, and keeps it alive until the call returns. The address,

rather than the aggregate bytes, is assigned by the GENERAL procedure. If GENERAL registers are exhausted, the stack argument slot contains that address; it does not contain the aggregate itself.

The copy gives the callee an addressable object with the declared aggregate type. A callee may modify its private copy without modifying the caller's source object. Baseline aggregates containing bit-fields and mixed integer/floating aggregates use the same INDIRECT rule; their member composition never changes argument registers. An extension aggregate with packed or unaligned members cannot cross this external ABI boundary unless a separate extension ABI permits it.

5.9 Variadic Calls

Fixed named arguments use the ordinary assignment procedure. Every unnamed argument is first subject to the C default argument promotions and is then placed in a 16-byte stack slot. In particular, `float` becomes `double`, and `_Bool`, `char`, `short`, and their signed or unsigned variants promote to `int` when `int` can represent all values of the source type. `long double` remains the Bedrock binary64 `long double` type. An aggregate slot contains the address of the caller-owned by-value copy described above.

The baseline `va_list` representation is one 64-bit pointer to the next unnamed argument slot. `va_start` initializes it to the first unnamed slot after any stack-assigned named arguments. `va_arg` reads the promoted representation from the low bytes of the current slot, or follows the copy address for an aggregate, and then advances the pointer by 16 bytes. `va_copy` copies the pointer value and `va_end` has no machine effect.

An unprototyped call applies the same promotions and places every argument on the stack. As required by C, the callee type must be compatible with the promoted types.

6 FREESTANDING C BINDING

This section defines the compiler-runtime and ELF linkage obligations of the freestanding C profile. It does not define process startup or hosted-library behavior.

6.1 Execution Boundary

Process entry belongs to the platform/runtime environment ABI. BSS zero-fill, object loading, and relocation belong to the Bedrock ELF ABI. Hosted library behavior is outside the freestanding C ABI.

6.2 Compiler Runtime Helpers

The compiler lowers addition, subtraction, negation, bitwise operations, comparisons, truncation, and extension of 128-bit integers without an implicit runtime call. It may call one of the helpers below for 128-bit multiplication, division, remainder, and shifts; conversion between a 128-bit integer and a supported floating-point type; or complex multiplication and division.

Every compiler-runtime helper is an ordinary `bedrock-c` function. Its arguments and results therefore follow Section 5, including the `R1:R0` return convention for `__int128`.

The tables below are the complete baseline helper set. A conforming compiler shall not make an implicit reference to another compiler-rt or libgcc helper unless a separate target extension defines that symbol and its ABI. A runtime may export additional symbols, but their presence does not enlarge the baseline contract.

In the signature column, `i128` means `__int128`, `u128` means `unsigned __int128`, `c32` means `float _Complex`, and `c64` means `double _Complex`. The signatures use ordinary C parameter and result classification; a shift count has type `int`.

Table 6-1. `__int128` Arithmetic and Shift Helpers

Operation	Helper	Signature
Multiply	`__multi3`	`i128(i128, i128)`
Signed divide	`__divti3`	`i128(i128, i128)`
Unsigned divide	`__udivti3`	`u128(u128, u128)`
Signed remainder	`__modti3`	`i128(i128, i128)`
Unsigned remainder	`__umodti3`	`u128(u128, u128)`
Shift left	`__ashlti3`	`i128(i128, int)`
Arithmetic shift right	`__ashrti3`	`i128(i128, int)`
Logical shift right	`__lshrti3`	`u128(u128, int)`

Table 6-2. `__int128` and Floating-Point Conversion Helpers

Conversion	Helper	Signature
Signed to binary32	`__floattisf`	`float(i128)`
Signed to binary64	`__floattidf`	`double(i128)`
Unsigned to binary32	`__floatuntisf`	`float(u128)`
Unsigned to binary64	`__floatuntidf`	`double(u128)`
Binary32 to signed	`__fixsfti`	`i128(float)`
Binary64 to signed	`__fixdfti`	`i128(double)`
Binary32 to unsigned	`__fixunssfti`	`u128(float)`
Binary64 to unsigned	`__fixunsdfti`	`u128(double)`

Table 6-3. Complex Arithmetic Helpers

Operation	Helper	Signature
Binary32 multiply	`__mulsc3`	`c32(float, float, float, float)`
Binary32 divide	`__divsc3`	`c32(float, float, float, float)`
Binary64 multiply	`__muldc3`	`c64(double, double, double, double)`
Binary64 divide	`__divdc3`	`c64(double, double, double, double)`

The four scalar arguments to a complex helper are the real and imaginary parts of its two operands, in operand order. Bedrock `long double` has the binary64 representation and call classification, so conversions involving `long double` use the corresponding `df` helper and `long double _Complex` multiplication or division uses the corresponding `dc3` helper.

6.3 Pointers and External Objects

A C function pointer is a 64-bit pre-segment near-call address. Code and data pointers have the same size and alignment, but remain distinct C type categories; neither representation creates an interchangeable code/data address space.

Far data and function pointers use their distinct 16-byte representations. Externally visible far functions have one canonical far-call entry address. Near and far entries use distinct symbols and addresses.

A common symbol has the ABI alignment of its declared type. An external object retains its declared alignment up to the 16-byte aggregate maximum. Stronger placement requirements belong to an explicit extension or platform contract.

6.4 Calls, Relocations, and Relaxation

The relocation records the linkage path selected for a C call:

Table 6-4. C Call Relocation Quick Reference

Call form	Relocation
Direct C call	`R_BEDROCK_CALL32S`
External call through the PLT	`R_BEDROCK_PLT32S`
Far symbol address word	`R_BEDROCK_FAR_ADDR64`
Far symbol segment word	`R_BEDROCK_FAR_SEGMENT64`
External far call slot	`R_BEDROCK_FAR_JUMP_SLOT`

The assembler and linker may relax an instruction sequence, but relaxation must preserve the C call interface, symbol identity, and observable control transfer defined by the original relocation. Relaxation is not a distinct C calling convention.

For a direct far call, the caller materializes `SEG(S)` and `FARADDR(S)` and transfers with `LCALL`. An indirect call loads both words from the function pointer; a segment image already held in a GS register is copied to the Rn operand required by `LCALL`. An external dynamic call loads the eagerly resolved, immutable 16-byte entry from `.got.far`.

6.5 Near/Far ABI Veneers

The current draft source profile diagnoses an ordinary/far function-pointer cast and does not require automatic veneer synthesis. A cast is not an implicit request for a veneer. This subsection instead defines the ABI of a veneer supplied through a separately named implementation extension or by runtime code.

Only a compiler with complete, prototyped function type information may synthesize a cross-convention veneer. The linker may redirect or deduplicate an already typed veneer but may not infer one.

- A near-to-far veneer receives `CALL`, performs `LCALL` to the far entry, and returns to the near caller with `RET`.
- A far-to-near veneer receives `LCALL`, performs a near `CALL` to the body, and returns to the far caller with `LRET`.

- Rn, floating-point, and `GS2`–`GS5` register arguments are forwarded without reclassification. Every stack argument is copied or repositioned because the return-frame sizes differ.
- Automatic veneers for variadic or unprototyped functions are forbidden.

6.6 TLS and OS ABI Boundary

TLS and OS boundary rules are supplied by the ABI specification.

7 C MEMORY MODEL

This section defines how the baseline C memory model maps to Bedrock memory operations and ordering instructions.

7.1 Ordinary Access Atomicity

The following guarantees apply to ordinary memory accesses. They do not by themselves establish C inter-thread ordering:

Table 7-1. Ordinary Memory Access Guarantees

Access	Guarantee
Aligned 1/2/4/8-byte load or store	tear-free
Access of 16 bytes or more	not atomic
Unaligned access	no tear-free guarantee

7.2 Atomic Object Eligibility

The native lock-free set consists of integer objects and ordinary 64-bit pointer objects whose size is 1, 2, 4, or 8 bytes and whose address satisfies the natural alignment for that size. An unaligned atomic object is not lock-free. Accordingly, `__atomic_always_lock_free` returns true only for those naturally aligned sizes.

The baseline accepts an atomic object only when it belongs to that native lock-free set. Any other atomic object—including a floating-point, aggregate, far-pointer, 128-bit integer, unsupported-width, or insufficiently aligned object—is a target constraint violation.

7.3 Native Atomic Primitives

Once the object satisfies the width and alignment requirements above, the compiler uses the following primitive lowering. The ordering sequences are defined separately below.

Table 7-2. Native Atomic Primitive Quick Reference

C operation	Bedrock primitive
Atomic load or store	ordinary aligned tear-free access plus the required ordering sequence
Compare-exchange	width-matched `CMPXCHG`
Exchange	compare-exchange loop using `CMPXCHG`

7.4 C Atomic Memory Order Mapping

Table 7-3. C Atomic Memory Order Mapping

C order	Bedrock order or sequence
relaxed	`relaxed`
consume	`acquire`
acquire	`acquire`
release	`release`
acq_rel	`acqrel`
seq_cst	`seqcst`

For C compare-exchange, the compiler selects the join of the success and failure orders; it does not compare their numeric encodings. Consume is treated as acquire, acquire joined with release is acquire-release, and sequential consistency dominates every other order. A failure performs no write. Any release component introduced by the joined instruction therefore creates neither a write nor a release sequence on failure, while a failed `CMPXCHG.SEQCST` still participates in the global SC order as an SC load.

Table 7-4. Atomic Load and Store Ordering

Operation	Lowering
Relaxed load/store	no fence
Consume or acquire load	load, then `AFENCE`
Release store	`AFENCE`, then store
Sequentially consistent load	`AFENCE`, load, `AFENCE`
Sequentially consistent store	`AFENCE`, store, `AFENCE`

A relaxed C thread fence emits no instruction. Consume, acquire, release, acquire-release, and sequentially consistent thread fences emit `AFENCE`.

7.5 Volatile, Device, and Executable Memory

Volatile accesses preserve the observable access count and order but do not themselves provide inter-thread or device ordering. MMIO ordering belongs to platform/runtime mapping attributes. Dynamic code patching requires `__bedrock_sync_cache` or a platform interface with equivalent cache synchronization; a relocation agent must synchronize patched instructions before transferring control to them.

7.6 C Fetch RMW Lowering

Table 7-5. C Fetch RMW Lowering

C operation	Lowering
`atomic_fetch_add`	`FETCHADD`
`atomic_fetch_sub`	`FETCHSUB`
`atomic_fetch_and`	`FETCHAND`
`atomic_fetch_or`	`FETCHOR`
`atomic_fetch_xor`	`FETCHXOR`

8 CALLING CONVENTION EXAMPLES

The first four cases are inputs to the executable reference model and illustrate the assignment procedure. The final two examples are generated output from the Bedrock LLVM backend. Examples do not introduce exceptions to the normative rules above.

8.1 Independent Register Classes and Pair Alignment

```
struct pair { uint64_t lo, hi; };
uint64_t mixed(uint64_t count, double scale,
  unsigned __int128 wide, struct pair pair, float bias);
```

The general cursor begins at 0 and the floating-point cursor begins at 0. `count` consumes `R0`; `scale` independently consumes `F0`. The GENERAL-PAIR argument rounds the general cursor from 1 to 2, so `wide` uses `R3:R2` and `R1` remains unused. The address of the caller-owned `pair` copy uses `R4`, and `bias` uses `F1`. No stack argument area is required.

8.2 sret Changes Ordinary Argument Assignment

```
struct triple { uint64_t x, y, z; };
struct triple build(uint64_t tag, signed __int128 wide,
  double factor);
```

Because the result is 24 bytes, its buffer address occupies `R0` and the general cursor begins at 1. Thus `tag` uses `R1`, `wide` uses `R3:R2`, and `factor` uses `F0`. The callee stores the result through the entry value of `R0` and returns the same address in `R0`.

8.3 A Pair That Does Not Fit Exhausts Its Class

```
void pressure(uint64_t a0, uint64_t a1, uint64_t a2,
  uint64_t a3, uint64_t a4, uint64_t a5, uint64_t a6,
  unsigned __int128 wide, uint64_t tail);
```

Arguments `a0` through `a6` use `R0` through `R6`. The next even pair would begin past `R7`, so `wide` is placed completely at `[SP+16]` and the GENERAL class becomes exhausted. `tail` therefore uses `[SP+32]`; it does not backfill `R7`.

8.4 Far Pointer Address and Segment Channels

```
far_data_pointer bind(uint64_t tag, far_data_pointer data,
  far_function_pointer function);
```

`tag` uses `R0`. The address of `data` uses `R1` and its validated image uses `GS2`; the address of `function` uses `R2` and its image uses `GS3`. No even GPR alignment is applied. The returned far pointer uses address register `R0` and segment register `GS1`.

8.5 Variadic Promotions and Slot Traversal

```
int trace(const char *format, ...);
trace("example", (signed char)-1, (float)1.5, pair);
```

The named `format` argument uses `R0`. The signed character is promoted to `int` and stored

at [SP+16]; the float is promoted to double and stored at [SP+32]. The caller creates the aggregate copy and stores its address at [SP+48]. A va_list initialized by va_start visits those three slots in that order by adding 16 after each access.

8.6 Generated Assembly Conventions

The assembly examples are non-normative. They were emitted with -Oz by Bedrock LLVM 22.1.8 and then checked against the call-assignment model, register-preservation contract, and the architectural semantics of CALL, RET, PUSHP, and POPP. File directives, LLVM basic-block comments, and local labels that do not affect the shown control flow are omitted. The scalar leaf example also replaces a one-instruction REPG body with the architecturally equivalent single-instruction REP spelling; otherwise instruction order and operands are unchanged. If an implementation sequence conflicts with a normative rule, the rule controls.

The benchmark sources use GNU C89 implicit return types. The declarations below show their equivalent explicit C types so that the ABI interface is unambiguous.

8.7 Scalar Leaf Function

```
int sum(int *p, int n);
```

On entry, p is in R0 and the sign-extended value of n is in R1. The generated function uses only volatile registers, does not change SP, and returns the integer sum in R0.

```
sum:
  mov.q  r0, r2
  clr.q  r0
  maxs.q r0, r1
  rep  r1, add.l [r2++], r0
  ret
```

Figure 8-1. Generated scalar leaf function

8.8 Non-Leaf Preservation and Call Alignment

```
int mf_pipeline(int *tmp, int *dst, int *src, int n,
  int ca, int cb, int cc);
```

The seven arguments occupy R0 through R6 in source order. The generated function keeps n, dst, tmp, and the first intermediate result live in nonvolatile R8 through R11. It therefore saves and restores both canonical register pairs.

```
mf_pipeline:
  pushp  3
  pushp  2
  sub.q  8, sp
  mov.q  r3, r8
  mov.q  r1, r9
  mov.q  r0, r10
  mov.q  r2, r1
  mov.q  r8, r2
  mov.q  r4, r3
  mov.q  r5, r4
  mov.q  r6, r5
  call  mf_fir_three
  mov.q  r0, r11
  mov.q  r9, r0
  mov.q  r10, r1
  mov.q  r8, r2
  call  mf_scale_store
  add.l  r11, r0
  extzq.l r0, r2
  mov.q  r9, r0
  mov.q  r8, r1
  add.q  8, sp
  popp  2
  popp  3
  jmp  mf_bias_sum
```

Figure 8-2. Generated non-leaf function with a tail transfer

Each `PUSHP` saves 16 bytes, so the pair saves leave the entry alignment unchanged. The additional 8-byte subtraction establishes `SP mod 16 = 8` before each `CALL`; the pushed return address therefore gives the callee a 16-byte-aligned entry stack. The epilogue removes the padding and restores `R10:R11` before `R8:R9`. The final `JMP` is a legal tail transfer because the frame and nonvolatile state have already been restored, the outgoing arguments have the ordinary ABI layout, and the target's `R0` result is also the result required by `mf_pipeline`.

9 SYSTEM INTERFACE BOUNDARY

The C ABI does not define a hosted process-entry protocol or system-call ABI. Those contracts belong to the active OS ABI.

9.1 SYSCALL Boundary

Instruction: SYSCALL

Ordinary C code reaches SYSCALL only through OS ABI wrappers or target-specific runtime code.

9.2 OS ABI Segment Policy

Segment initialization and process policy are defined by the active OS ABI.